

- RAG Learning

1.2 Software and Software Development

Computer Science — OCR — Component 01 (Computer Systems)

1.2.1 Systems Software

Purpose of an Operating System

An **operating system (OS)** is a collection of system software that manages hardware resources, provides common services for application programs, and acts as an **abstraction layer** between user software and the underlying machine. Examples: Windows, macOS, Linux, Android, iOS.

At A-Level, an OS's purpose is usually described under three umbrella ideas:

- **Abstraction** — hides hardware complexity behind uniform interfaces (e.g. a single `open()` call works for any file on any disk).
- **Resource management** — arbitrates access to CPU, memory, storage and peripherals between competing processes.
- **Security & user management** — authenticates users, enforces file permissions, isolates processes, controls I/O privileges.

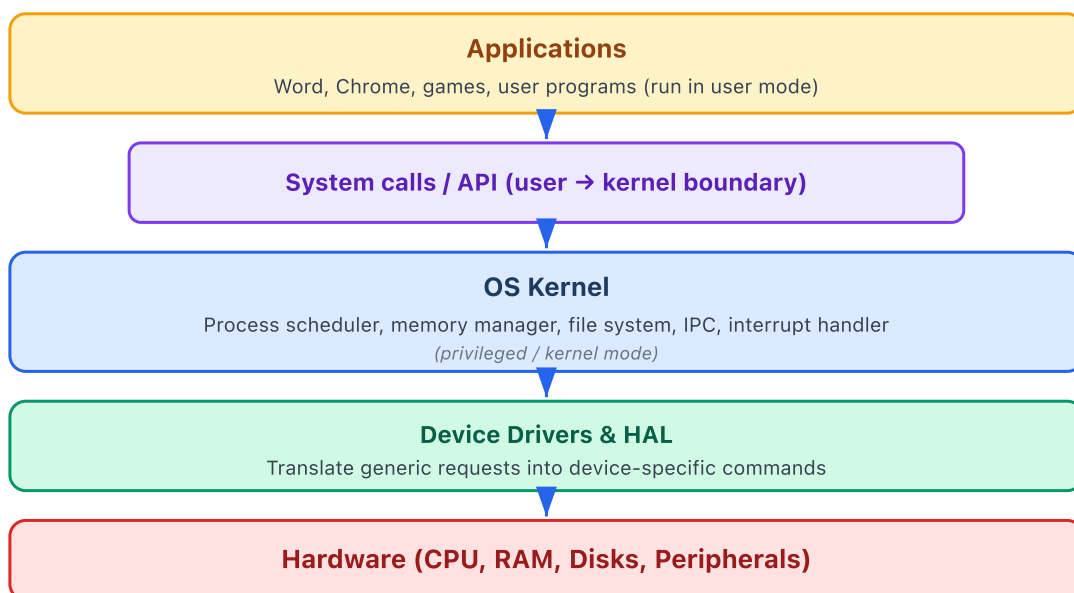
Secondary purposes include providing a **user interface** (CLI or GUI), maintaining a **file system**, handling **error detection and recovery**, and providing **utilities** for housekeeping tasks such as disk management.

Components of an Operating System

Component	Role
-----------	------

Kernel	The core of the OS. Runs in privileged (kernel) mode; manages processes, memory, interrupts, and I/O. All other components rely on it.
Shell	Outer layer that communicates with the user — CLI (Bash, PowerShell, cmd) or GUI (Explorer, Finder, GNOME). Translates user commands into kernel calls.
Device drivers	Small programs that translate generic OS I/O requests into the specific commands a particular piece of hardware understands. One driver per device class (printer driver, NIC driver, GPU driver).
HAL (Hardware Abstraction Layer)	A thin layer between the kernel and device drivers that presents a uniform hardware interface. Lets the same kernel binary run on different motherboards/CPUs.
Utilities	Support programs bundled with the OS (defrag, antivirus, file manager). See 1.2.1 Utility Software.

Operating System Layers



Exam Tip: A common misconception is that "the OS" and "the kernel" are synonymous. The **kernel** is only the core component; the OS is the kernel *plus* the shell, drivers, libraries and utilities. Saying "the kernel handles everything" in an exam will lose nuance marks.

Memory Management

Most modern OSes give every process its own **virtual address space** — the process sees a flat, contiguous region of memory, but physically its bytes are scattered across RAM (and possibly disk). The three key techniques are **paging**, **segmentation** and **virtual memory**.

Paging

Paging divides physical RAM into fixed-size blocks called **frames** (typically 4 KiB) and divides each process's virtual address space into equally sized **pages**. A **page table** per process maps virtual pages to physical frames.

- Fixed-size → easy to allocate, no external fragmentation.
- Suffers from **internal fragmentation** (last page usually partially filled).
- Page table is consulted on every memory access; caches called **TLBs** speed this up (not examinable detail, but useful context).

Segmentation

Segmentation divides a program into **variable-size logical segments** that correspond to program structure (code, stack, heap, individual procedures). Each segment has a base address and a length.

- Matches the programmer's view — protect the code segment as read-only, the stack as non-executable.
- Suffers from **external fragmentation** because segments are different sizes and leave awkward gaps when freed.

Virtual Memory

Virtual memory extends apparent RAM by using part of secondary storage (a **swap file** / **page file**) to hold pages that do not fit in physical RAM. When a process accesses a page that is on disk, a **page fault** is raised; the OS suspends the process, pages in the needed block (evicting another via e.g. LRU), then resumes execution.

- Lets programs exceed installed RAM and lets many processes coexist.
- If the working set is much bigger than RAM, the system spends most of its time paging — this pathological state is called **thrashing**.
- Disk access is $\sim 10^5\times$ slower than RAM, so frequent page faults devastate performance.

Exam Tip: Do not confuse **virtual memory** (swap-to-disk technique) with **virtual address space** (the logical view each process has of memory). Exam answers often demand all three terms: *paging, page fault, page table*.

Interrupts and Interrupt Handling

An **interrupt** is a signal to the CPU from hardware or software indicating that an event needs immediate attention. The CPU suspends what it is doing, saves its state, runs an **Interrupt Service Routine (ISR)**, then resumes the interrupted program.

Type	Source	Example
Hardware interrupt	Raised by a device via the control bus (IRQ line)	Key press, mouse move, disk-transfer complete, clock tick, power failure
Software interrupt (trap)	Raised by an executing instruction	System call, divide-by-zero, page fault, illegal instruction

The Interrupt Cycle

1. At the end of every FDE cycle the CPU **checks the interrupt register** for pending interrupts.
2. If one is present and its **priority** exceeds that of the current task, the CPU pushes the PC, registers and flags onto the **stack**.
3. The PC is loaded with the address of the relevant **ISR** (from the interrupt vector table).
4. The ISR executes, communicating with the device.
5. On **RTI** (return from interrupt), the saved state is popped back and the original task resumes transparently.

Interrupt Priority

Each interrupt source is assigned a priority level. A low-priority ISR can itself be interrupted by a higher-priority one (nested interrupts), but not by an equal or lower one. This ensures critical events (e.g. power fail) always get serviced first.

Polling vs Interrupt-Driven I/O

Approach	How it works	Pros	Cons
Polling	CPU repeatedly checks the device's status register in a loop	Simple, deterministic	Wastes CPU cycles when device is idle; poor for many devices
Interrupt-driven	Device raises an IRQ; CPU only services it when signalled	CPU can do useful work while waiting; scales to many devices	More complex hardware; ISR must be short

Exam Tip: For "why are interrupts used?" questions, always contrast with polling: interrupts free the CPU to do other work, whereas polling wastes cycles constantly checking a status flag.

Scheduling

Scheduling is the mechanism by which the OS decides which ready process gets the CPU next, and for how long. The aim is to maximise throughput, minimise response time, and keep the system fair.

Algorithm	How it works	Notes
Round Robin (RR)	Each process gets a fixed time slice (<i>quantum</i>) in turn; preempted when the quantum expires.	Fair, good response time; needs a well-chosen quantum (too short → overhead; too long → sluggish).
First Come First Served (FCFS)	Runs processes in arrival order to completion; non-preemptive.	Simple but suffers from the <i>convoy effect</i> — one long job

		blocks many short ones.
Shortest Job First (SJF)	Picks the ready process with the shortest expected CPU burst; non-preemptive.	Minimises average waiting time but can starve long jobs; burst length must be estimated.
Shortest Remaining Time (SRT)	Preemptive SJF — if a newly arrived job is shorter than what's left of the current job, switch.	Even better throughput than SJF but higher overhead and still can starve long jobs.
Priority scheduling	Each process has a priority; highest-priority ready process runs.	Good for real-time. Low-priority jobs can starve — mitigated by <i>aging</i> (gradually raise priority of waiting jobs).
Multi-level Feedback Queues (MLFQ)	Multiple queues at different priorities; a job that uses its full quantum drops to a lower queue; a job that blocks on I/O stays high.	Adaptive — favours interactive jobs, penalises CPU hogs. Used (in practice) by most modern OSes.

Exam Tip: If a question asks which scheduler is best for a desktop OS, the answer is usually **multi-level feedback queues** or **round robin** — both give good response time to interactive users.

Types of Operating System

Type	Description	Example
Distributed	A single OS spanning many networked machines, sharing workload and presenting a unified image to the user.	Google's internal clusters, Plan 9, some supercomputers
Multi-user	Allows many users to log in and share resources simultaneously, each isolated from the others.	Linux servers, mainframes, UNIX

Multi-tasking	Allows a single user to run many programs apparently simultaneously by rapid CPU switching.	Windows, macOS, most desktop/mobile OSes
Real-time (RTOS)	Guarantees a response within a strict time limit; correctness depends on <i>when</i> as well as <i>what</i> .	Car ABS controllers, pacemakers, industrial robots (VxWorks, FreeRTOS)
Embedded	Specialised OS on a device with fixed function and limited resources; often real-time.	Washing machine firmware, router OS, smart meters

Virtual Machines

A **virtual machine (VM)** is a software emulation of a computer system that runs programs as if on a physical machine. Two common forms: **system VMs** (emulate a whole PC — e.g. VirtualBox, VMware, Hyper-V) and **process VMs** (execute *intermediate code* — e.g. JVM, .NET CLR).

Intermediate Code and JIT

Process VMs run programs compiled to a platform-neutral **intermediate code** (also called **bytecode** or p-code). The VM then either **interprets** the bytecode or uses a **just-in-time (JIT) compiler** to translate hot methods into native machine code at runtime, blending interpreted portability with compiled speed.

Advantages	Disadvantages
Portability — "write once, run anywhere" via intermediate code	Performance overhead vs native code (mitigated but not eliminated by JIT)
Isolation — a crashing VM cannot corrupt the host	VM and guest OS need memory/CPU themselves
Legacy software support — run old OSes on modern hardware	Hardware acceleration (GPUs, peripherals) may be restricted

Safer testing / sandboxing of untrusted code	Setup complexity; licensing complications for guest OS
Server consolidation — many logical servers on one physical box	Debugging is harder (extra translation layer)

Exam Tip: In H446 "intermediate code" usually refers to **bytecode** in a process VM context (Java, Python .pyc, C# IL). If the question says "intermediate representation", it can also mean the internal form produced during compilation — see 1.2.2.

Utility Software

Utility software performs housekeeping tasks that maintain, optimise or protect the computer system. It is classed as *systems software* because it manages the machine itself rather than solving user problems directly.

Utility	Purpose
Defragmenter	Rearranges file blocks on a magnetic disk so each file occupies contiguous sectors → fewer head movements → faster read. Not needed (and harmful) on SSDs.
Disk cleanup	Removes temporary files, cached installers, obsolete system logs to reclaim space.
Compression	Reduces file size using lossless algorithms (ZIP, 7z, gzip) for storage or transfer.
Backup	Creates copies of files on separate media. May be <i>full</i> , <i>incremental</i> (only changes since last backup) or <i>differential</i> (all changes since last full backup).
Antivirus / antimalware	Scans storage and memory for known malware signatures and suspicious behaviour; quarantines or deletes threats.

Encryption	Converts data to ciphertext (e.g. BitLocker, FileVault, VeraCrypt) so it is unreadable without the key.
File manager	Provides a UI for browsing, copying, renaming, searching files (Windows Explorer, Finder, Nautilus).
Device drivers	Supply the OS with the code needed to communicate with specific hardware.

Open Source vs Closed (Proprietary) Source Software

Open source software is distributed under a licence that gives users the right to *view, modify and redistribute* the source code (e.g. GPL, MIT, Apache). **Closed source (proprietary)** software ships only as a compiled binary; the source is a trade secret and the licence restricts modification and redistribution.

	Open source	Closed source
Cost	Usually free to obtain	Usually paid; licence fees per seat/user
Source access	Full source code available	No source; binary only
Modification	Permitted (and often required to be shared back)	Forbidden by licence / impossible without source
Support	Community forums, optional paid support (Red Hat model)	Dedicated vendor support, SLAs
Security	"Many eyes" argue more secure; but bugs are also public	Security through obscurity; patches rely on vendor
Quality & features	Variable — depends on community size	Professional QA, coordinated UX

Examples	Linux kernel, Firefox, LibreOffice, Python, Blender	Windows, macOS, Microsoft Office, Adobe Photoshop
-----------------	---	---

Exam Tip: Do not equate "open source" with "free" (as in price). Open source licences grant *freedom* (the 4 freedoms: run, study, modify, redistribute). Some commercial open-source products (Red Hat Enterprise Linux) are sold for money while remaining open source.

1.2.2 Applications Generation

Turning human-readable source code into an executable program involves a toolchain: **translators**, **linkers**, **loaders** and **libraries**. This section examines each in A-Level depth.

Translators: Compilers, Interpreters, Assemblers

A **translator** is a program that converts code from one language to another. The three types are **assembler** (assembly → machine code, 1:1), **compiler** (high-level → machine/intermediate code, ahead of time) and **interpreter** (reads and executes source line by line).

	Compiler	Interpreter	Assembler
Input	High-level language	High-level (or bytecode) language	Assembly mnemonics
Output	Executable object/machine code (produced once)	None — interpreter <i>executes</i> as it reads	Object/machine code (1 asm instruction → 1 machine instruction)
Speed of execution	Fast — native code, optimised	Slow — re-parsed each run	Very fast

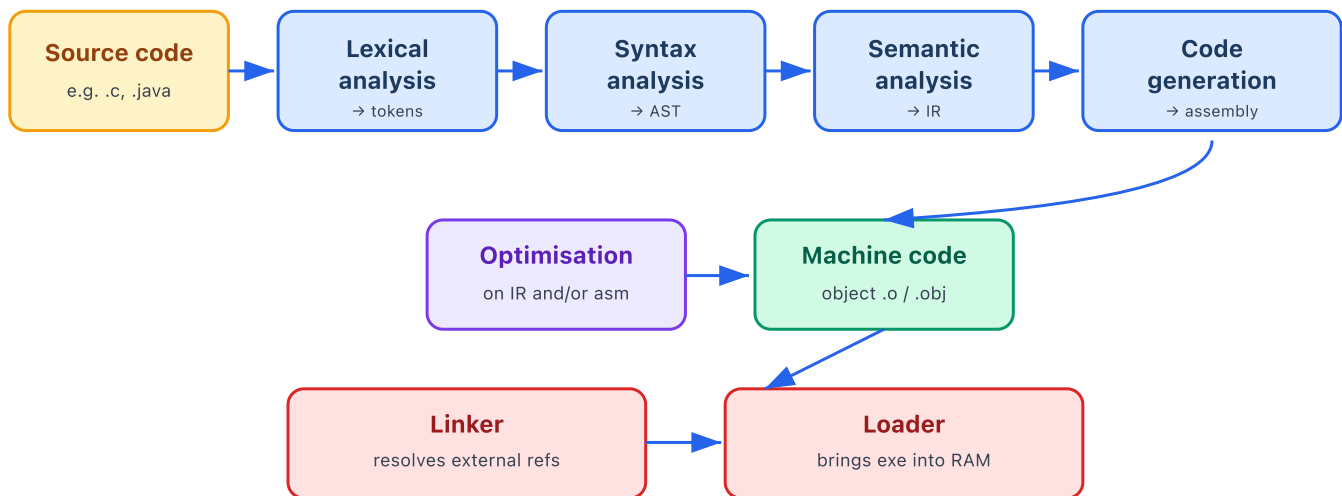
Error reporting	Full list after compile; program will not run if any error	Stops at first error; useful for development	Line-by-line
Portability	Must recompile for each target	Source runs anywhere with the interpreter	Target-specific
Source distribution	Can ship compiled binary only (protects IP)	Must ship source (or bytecode)	Binary distribution
Memory usage at run-time	Just the executable	Executable + interpreter in memory	Just the executable
Typical examples	C, C++, Rust, Go	Python, JavaScript, PHP, BASIC	GNU as, MASM, NASM

Exam Tip: A favourite exam trap: candidates confuse an **interpreter** with a **compiler**. The distinction is: an *interpreter translates AND executes line by line*; a *compiler only translates* — a separate step runs the result. Always state both jobs of the interpreter to secure marks.

Stages of Compilation

A compiler is not a monolith — it is a pipeline of distinct phases. The H446 specification explicitly names four: **lexical analysis**, **syntax analysis**, **code generation** and **optimisation** (semantic analysis, below, is extra detail often shown between syntax analysis and code generation).

Compilation Pipeline



1. Lexical Analysis (the lexer / tokenizer)

Lexical analysis reads the raw source character by character and groups characters into **tokens** — keywords, identifiers, literals, operators, punctuation. Whitespace and comments are discarded. Identifiers are entered into the **symbol table**.

Source: `total = price * 1.2    // add VAT`

Tokens: `IDENT("total") ASSIGN IDENT("price") MUL  NUMBER(1.2)`

Symbols: `total → (type: real, scope: local)`
 `price → (type: real, scope: local)`

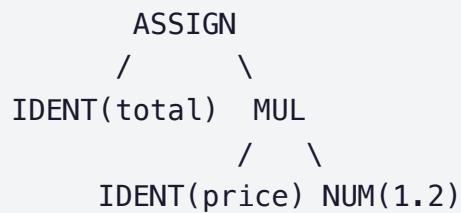
- Output: a stream of tokens plus a partially-built symbol table.
- Detects errors such as *invalid characters* or *malformed numeric literals*.

2. Syntax Analysis (the parser)

Syntax analysis takes the token stream and verifies it matches the language's grammar (usually expressed in BNF). It builds an **Abstract Syntax Tree (AST)** — a tree representing the hierarchical structure of the program.

Tokens for: `total = price * 1.2`

AST:



- Detects **syntax errors** (missing semicolons, unmatched brackets, illegal expressions).
- The AST is the canonical input to all later phases.

3. Semantic Analysis

Semantic analysis walks the AST and checks it makes *sense* — type compatibility, variable declared before use, correct number of arguments, scope rules. It annotates the tree with type information and produces an **intermediate representation (IR)**, a simplified, machine-independent form.

Semantic errors (e.g. adding a string to an integer, calling an undeclared function) are reported here. The IR is typically a linear three-address-code form such as `t1 = price * 1.2; total = t1;` which is easy to optimise.

4. Code Generation & Optimisation

Code generation converts the IR into target **machine code** (or assembly, then assembled). **Optimisation** improves the code without changing its behaviour — faster, smaller, or less power-hungry.

Common optimisations include:

- **Constant folding** — evaluate constant expressions at compile time (`2*3.14*r` → `6.28*r`).
- **Dead code elimination** — remove code that cannot affect output.
- **Loop unrolling** — replicate the loop body to reduce branch overhead.
- **Register allocation** — keep hot variables in CPU registers rather than RAM.

- **Common sub-expression elimination** — compute `a*b` once if it appears twice.

Exam Tip: H446 markschemes expect you to name the stages *in order* and describe the output of each: lexical → tokens; syntax → AST; semantic → IR / type-checked tree; code-gen → machine code. A useful mnemonic: "**Lexer Parses Semantics then Generates**".

Linkers and Loaders

A **linker** combines one or more **object files** (the compiled output of each source file) together with library code to produce a single executable, resolving symbolic references between them. A **loader** is the OS component that copies that executable from disk into RAM and hands control to its entry point when you run the program.

Static vs Dynamic Linking

	Static linking	Dynamic linking
When it happens	At compile-time: library code is copied into the executable	At load/run-time: the OS maps the library into the process's address space
Executable size	Large — contains all library code	Small — only references to external DLL/.so
RAM usage	Each process has its own copy of the library	One shared copy in RAM used by all processes (shared libraries)
Updates	Must recompile/relink whole program to get new library version	Replace the DLL/.so — all programs benefit immediately
Portability	Self-contained — just ship the executable	Missing / wrong-version libraries cause "DLL hell"
Typical extension	<code>.a</code> (UNIX), <code>.lib</code> (Windows)	<code>.so</code> (UNIX), <code>.dll</code> (Windows), <code>.dylib</code> (macOS)

Libraries

A **library** is a pre-written, pre-compiled collection of procedures, classes or data used by programs. Libraries promote code reuse and let developers stand on well-tested foundations rather than reinventing common algorithms.

Benefits: faster development, fewer bugs (mature code), consistent behaviour across programs, smaller source codebase.

Tradeoffs: dependency on third-party code (licensing, abandonment risk), compatibility problems between versions, potentially unused code bloats the final binary (if statically linked), security surface grows (log4shell-style vulnerabilities).

1.2.3 Software Development Methodologies

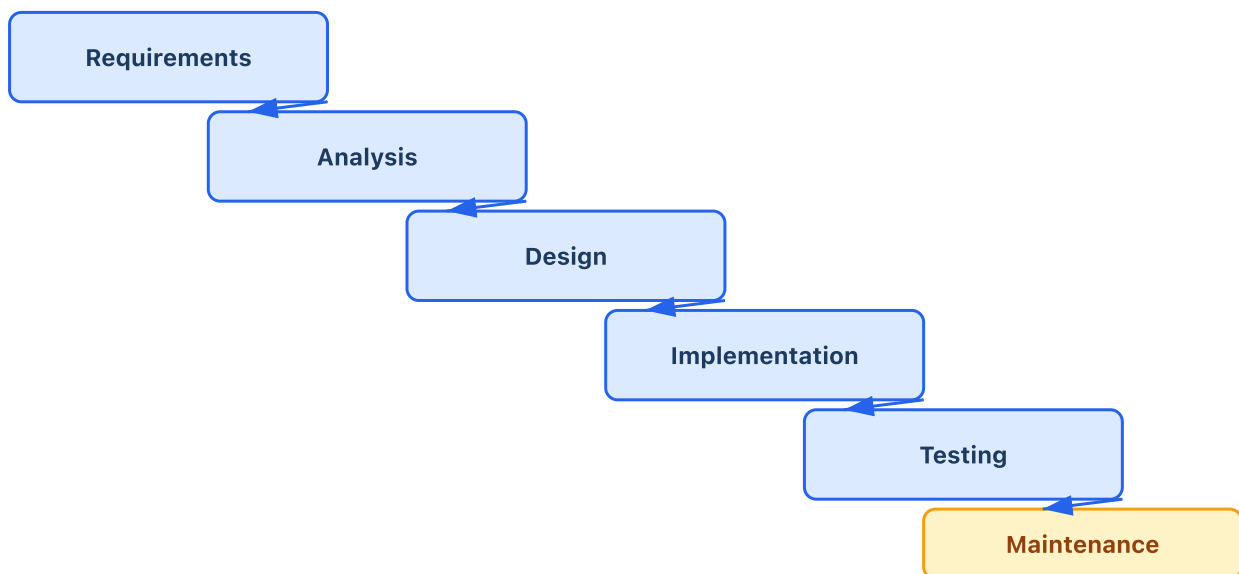
A **software development methodology** is a structured framework used to plan, manage and control the creation of software systems. The choice of methodology affects how requirements are gathered, how work is divided, how risks are managed, and how (and when) the customer sees progress.

Waterfall

The **Waterfall** model is a linear, sequential approach. Each stage is fully completed and signed off before the next begins, and (in its strict form) you do not revisit earlier stages.

1. **Requirements** — gather and document what the system must do.
2. **Analysis** — model the problem domain; produce formal specifications.
3. **Design** — system and detailed design documents; architecture, data structures.
4. **Implementation** — programmers write the code.
5. **Testing** — verify against requirements (unit, integration, system, acceptance).
6. **Maintenance** — fix bugs and adapt to changing environment after release.

Waterfall Model



Advantages	Disadvantages
Simple to understand and manage; clear milestones	Very inflexible — late changes are expensive
Documentation heavy — good for handover	Customer only sees software near the end
Works well when requirements are fixed and well known	Bugs found late (in testing) can force major rework
Easy to contract and bill — each stage has deliverables	Assumes requirements can be fully known up front — often false

Best suited to: safety-critical, well-understood, regulated projects (medical devices, aerospace firmware, government systems) where requirements are frozen early.

Agile

Agile is an umbrella term for iterative methodologies (Scrum, XP, Kanban, etc.) that break development into short cycles (**sprints**, typically 1–4 weeks), each producing potentially

shippable software. Requirements evolve through continuous collaboration with the customer.

Core Agile ideas from the Agile Manifesto:

- **Individuals and interactions** over processes and tools.
- **Working software** over comprehensive documentation.
- **Customer collaboration** over contract negotiation.
- **Responding to change** over following a plan.

User stories describe features from the user's viewpoint (*"As a [role] I want [goal] so that [benefit]"*). The **product backlog** is the prioritised list of all stories; each sprint pulls from the top of the backlog. **Continuous delivery** means the software is always in a releasable state.

Advantages	Disadvantages
Handles changing requirements gracefully	Less predictable cost/time — hard to fix-price
Customer sees working software early and often	Light documentation complicates handover and audit
Defects caught within each sprint	Needs a highly engaged customer — not always available
High team morale, responsive to feedback	Scope creep risk if backlog is not managed

Extreme Programming (XP)

Extreme Programming (XP) is an Agile methodology that stresses engineering practices for code quality: **pair programming**, **test-driven development (TDD)**, continuous integration, refactoring, collective ownership, small releases.

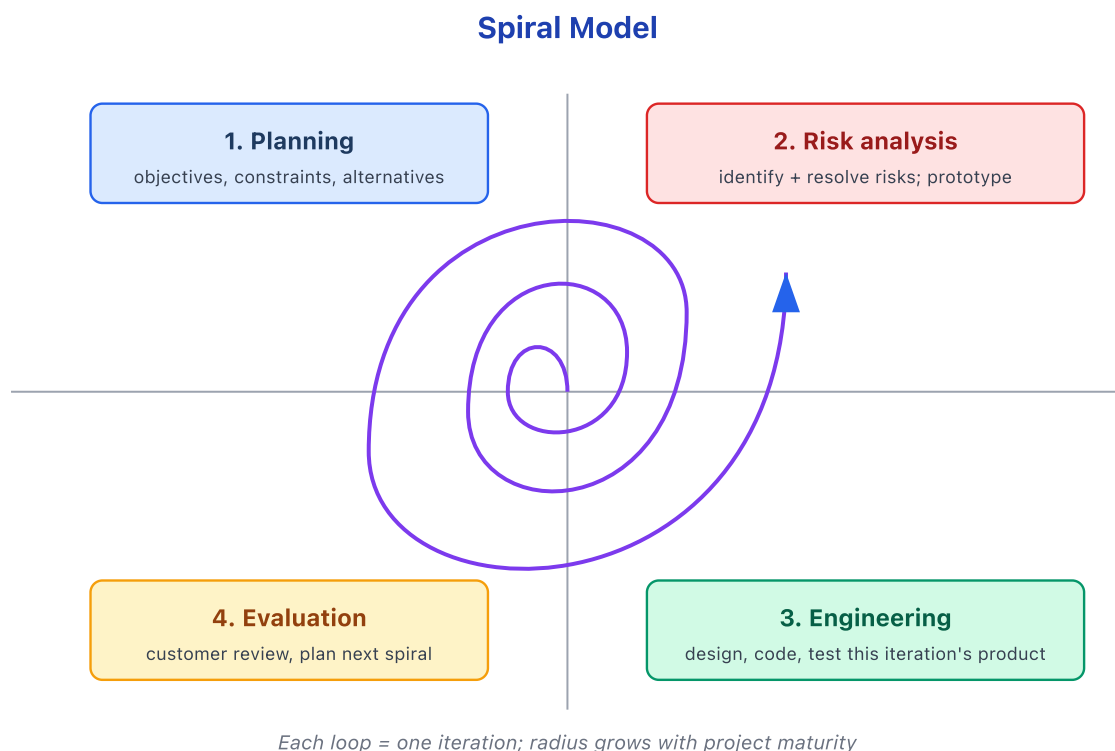
- **Pair programming** — two developers on one machine: one "driver" types, one "navigator" reviews and thinks strategically. Knowledge shared, fewer defects.

- **TDD** — write a failing unit test *before* writing code; implement just enough code to pass; then refactor. Cycle known as "red — green — refactor".
- **Continuous integration** — code is merged into mainline many times per day and built/tested automatically.

Best for: small co-located teams working on changing, poorly-understood requirements where code quality matters.

Spiral Model

The **Spiral model** (Boehm, 1986) is a **risk-driven**, iterative approach. Each loop through four quadrants (plan, analyse risks, engineer, evaluate) produces a more complete prototype. Expensive or risky issues are flushed out early by explicit risk analysis.



Advantages	Disadvantages
Explicit risk management — good for large, unknown projects	Overkill for small projects

Customer involved throughout	Expensive — repeated risk analysis, prototypes
Flexible — requirements can change between spirals	Requires skilled risk assessors; process is complex

Best for: large, complex, high-risk, long-term projects (defence, aerospace, novel research systems).

Rapid Application Development (RAD)

RAD emphasises **prototyping** and **user involvement** to deliver working software quickly. Developers build throw-away or evolutionary prototypes, show them to users, refine and repeat. Heavy use of 4GL tools, code generators and pre-built components keeps the cycle short.

- Particularly suited to UI-driven, data-centric business applications where look-and-feel needs rapid iteration.
- Less suitable for systems with strict performance or correctness requirements (real-time, safety-critical).
- Requires committed, available users who can give continuous feedback.

Comparison of Methodologies

Methodology	Style	Best when...	Weakness
Waterfall	Linear, document-led	Requirements are fixed and regulated	Inflexible; customer sees software late
Agile (Scrum)	Iterative, short sprints	Requirements evolve; engaged customer	Hard to fix-price; light docs
XP	Engineering-practice-heavy Agile	Quality critical; small, co-located team	Pair programming demands discipline

Spiral	Risk-driven, iterative	Large, risky, novel systems	Expensive; needs risk expertise
RAD	Prototyping-led	UI-heavy business apps; quick turnaround	Unsuitable for performance-critical work

Exam Tip: A common 6-mark question asks you to *justify* a methodology for a given scenario. Always match the scenario cues (changing requirements, safety-critical, short deadline, novel problem) to a methodology and explicitly explain why — don't just pick one and describe it.

Testing

Testing is the process of evaluating software to find defects and verify it meets its specification. A-Level H446 asks about *who* tests (alpha vs beta), *how* they test (black-box vs white-box), and *what granularity* (unit, integration, system, acceptance).

Type	Description
Alpha testing	Performed <i>internally</i> by the development organisation, often by staff who are not developers. Finds obvious bugs before external release.
Beta testing	Performed <i>externally</i> by a subset of real end-users on real hardware. Finds problems only visible at scale or across many configurations.
Black-box testing	Testers see only the specification — they choose inputs based on what the software <i>should</i> do. Good for validation against requirements.
White-box (glass-box) testing	Testers see the source code — choose inputs to exercise every branch, loop, condition. Measures coverage.
Destructive testing	Deliberately feeds invalid, extreme or malicious inputs to try to crash the software. Probes robustness.

Unit testing	Tests individual functions/classes in isolation, usually automated (JUnit, pytest).
Integration testing	Tests combined units to catch interface mismatches between modules.

Testing is typically layered: unit tests during implementation, integration tests as modules are combined, system tests on the whole build, then alpha → beta → release.

1.2.4 Types of Programming Language

Low-Level vs High-Level Languages

	Machine code	Assembly	High-level
Form	Binary opcodes executed directly by CPU	Mnemonic 1:1 with machine instructions	English-like, abstract
Translation needed	None	Assembler (1:1)	Compiler or interpreter (1:many)
Portability	None — tied to CPU family	None — tied to CPU family	Good — recompile for target
Productivity	Extremely low	Very low	High
Hardware control	Total	Total	Limited (needs libraries or inline asm)
Examples	Hex dumps, firmware	x86 asm, ARM asm, MIPS	Python, Java, C#, C++

Low-level languages (machine code, assembly) are close to hardware — each instruction maps (nearly) 1:1 to a CPU operation. They give maximum control and speed but are unreadable, unportable, and slow to write. **High-level languages** abstract away the CPU,

giving readable, portable, expressive code at the cost of some run-time performance and hardware control.

Assembly Language

Assembly uses **mnemonics** — short human-readable codes for each machine instruction. The H446 specification requires familiarity with the Little Man Computer (LMC) instruction set (ADD, SUB, STA, LDA, BRA, BRZ, BRP, INP, OUT, HLT, DAT). The ARM-style mnemonics below are a general illustration, not the OCR set:

Mnemonic	Meaning	Example
MOV	Move / load immediate into register	MOV R1, #5 — put literal 5 in R1
LDR	Load from memory	LDR R1, 100 — load value at address 100 into R1
STR	Store to memory	STR R1, 100 — store R1's value at address 100
ADD	Add	ADD R1, R2, #3 — $R1 = R2 + 3$
SUB	Subtract	SUB R1, R2, R3 — $R1 = R2 - R3$
CMP	Compare (sets SR flags)	CMP R1, #0 — compare R1 with 0
BEQ	Branch if equal (Z flag set)	BEQ end
BNE	Branch if not equal	BNE loop
B	Unconditional branch	B main
HALT	Stop execution	HALT

Labels are symbolic names for memory addresses, used as branch targets or data locations.

Operands can be registers, memory addresses or literal constants — the form they take defines the *addressing mode*.

```

; Count from 10 down to 0
      MOV    R1, #10      ; R1 = 10
loop:  CMP    R1, #0      ; compare R1 with 0
      BEQ    end        ; if zero, branch to end
      SUB    R1, R1, #1   ; R1 = R1 - 1
      B      loop       ; branch back
end:   HALT

```

Addressing Modes

An **addressing mode** specifies how the operand of an instruction is interpreted — whether it is a literal value, a memory address, a register holding an address, or a calculated address. H446 requires four modes: **immediate**, **direct**, **indirect** and **indexed**.

Mode	Notation	Meaning	Worked example (assume memory shown below)
Immediate	#n	Operand is the value	LDR R1, #4 → R1 = 4
Direct	addr	Operand is the memory address holding the value	LDR R1, 100 → R1 = memory[100] = 200
Indirect	[Rn] or (addr)	Operand is an address whose <i>contents</i> are the address of the value	LDR R1, (100) : memory[100]=200, so R1 = memory[200] = 99
Indexed	base, Rx	Effective address = base + index register's contents	LDR R1, 100, R2 with R2=2 → R1 = memory[102] = 7

Addressing Modes (Memory & Register Grid)

Immediate #4

operand is value itself

4 → R1 = 4

Direct 100

operand is address of value

R1 = mem[100] = 200

Indirect (100)

address points to address

mem[100]=200 → R1 = mem[200] = 99

Indexed 100, R2

base + index register

R1 = mem[100+2] = mem[102] = 7

Memory	
Addr	Value
100	200
101	42
102	7
...	
200	99

R2 (index)

value = 2

Same instruction bits, different meaning depending on mode flag

Exam Tip: Indirect addressing is the most misremembered mode. Phrase it as "the operand is the address of a location that itself *contains* the address of the data" — two hops through memory. Indexed addressing is how arrays are implemented: `arr[i]` becomes `base + i*element_size`.

Paradigms

A **programming paradigm** is a style of organising a program and its data. Languages often support multiple paradigms, but each paradigm makes certain patterns easier to express and optimise for.

Procedural

Procedural programming organises code as a sequence of **procedures** (functions) that operate on **variables**. Control flows through **sequence**, **selection** (if / case) and **iteration** (while / for). Structured programming banishes `goto` in favour of nested block structures; each routine has one entry and (ideally) one exit.

```
FUNCTION area_of_circle(radius)
    pi = 3.14159
```

```
RETURN pi * radius * radius
ENDFUNCTION

a = area_of_circle(10)    ; call the procedure; it has no "owner"
PRINT a
```

Best for: small-to-medium programs with straightforward data flow — scripts, utilities, algorithmic code.

Object-Oriented

In **object-oriented programming (OOP)**, data and the code that acts on it are bundled into **objects**, instances of **classes**. Four pillars: **encapsulation, inheritance, polymorphism, abstraction**.

Pillar	Meaning
Encapsulation	Data (attributes) and behaviour (methods) bundled together; attributes made private and accessed via getters/setters. Protects invariants.
Inheritance	A subclass reuses and extends a parent class, inheriting its attributes and methods. "Is-a" relationship.
Polymorphism	The same method name behaves differently depending on the object's class (override) or argument types (overload).
Abstraction	Expose only the essential interface; hide implementation details behind it.

```
CLASS Animal
  PRIVATE name
  PRIVATE sound

  PUBLIC PROCEDURE NEW(n, s)      ; constructor
    name = n
    sound = s
  ENDPROCEDURE

  PUBLIC FUNCTION get_name()
```

```

    RETURN name
ENDFUNCTION

PUBLIC PROCEDURE speak()
    PRINT name + " says " + sound
ENDPROCEDURE
ENDCLASS

CLASS Dog INHERITS Animal                ; subclass
    PUBLIC PROCEDURE NEW(n)
        SUPER.NEW(n, "Woof")            ; call parent constructor
    ENDPROCEDURE

    PUBLIC PROCEDURE fetch()              ; new method only on Dog
        PRINT get_name() + " fetches the stick"
    ENDPROCEDURE
ENDCLASS

rex = NEW Dog("Rex")
rex.speak()      ; "Rex says Woof"    - inherited
rex.fetch()      ; "Rex fetches the stick" - own method

```

Best for: large systems with many entity types and evolving domain logic (games, business applications, GUIs, simulations).

Exam Tip: In OCR pseudo-code, use `INHERITS`, `SUPER`, `PRIVATE` / `PUBLIC` to flag visibility, and a `NEW` procedure as the constructor. Examiners credit the *concept* (encapsulation of name, inheritance of speak) — exact syntax is flexible.

Functional

Functional programming treats computation as the evaluation of mathematical functions. Emphasises **pure functions** (no side-effects), **immutability** (values are never changed, only new values created), **first-class functions** (functions passed as data), **higher-order functions** (map, filter, reduce), and **recursion** instead of loops.

```

; Sum of squares of even numbers, functional style:
nums      = [1, 2, 3, 4, 5, 6]

```

```
evens      = filter(lambda x: x MOD 2 == 0, nums)      ; [2, 4, 6]
squares    = map(lambda x: x * x, evens)               ; [4, 16, 36]
result     = reduce(lambda a, b: a + b, squares, 0)    ; 56
```

Best for: data transformation pipelines, concurrent/parallel code (no shared mutable state), symbolic computation. Examples: Haskell, F#, Lisp, Scheme. Full treatment lives in 2.1/2.2.

Declarative

Declarative programming describes *what* you want, not *how* to compute it. The runtime decides the execution strategy. Examples:

- **SQL** — `SELECT name FROM pupils WHERE grade = 'A'`. You specify the desired result set; the DBMS chooses indexes and join order.
- **Prolog** — logic-based. You declare facts (`parent(alice, bob).`) and rules (`ancestor(X,Y) :- parent(X,Y).`); Prolog's inference engine answers queries.

Best for: querying databases, theorem proving, constraint satisfaction, expert systems.

Event-Driven

In **event-driven programming**, program flow is determined by **events** (button clicks, key presses, network packets, timer ticks). The program sits in an **event loop** and dispatches each event to a registered **event handler** (callback).

Underpins almost all GUI frameworks (Windows Forms, WPF, Swing, JavaFX, Qt, Tkinter) and JavaScript in the browser. Best for: user interfaces and reactive systems. Easy to combine with OOP (each widget is an object with event handlers).

Choosing a Paradigm

Paradigm	Best suited to...
Procedural	Small algorithmic tasks, scripts, embedded firmware
Object-oriented	Large, extensible systems with rich domain models (games, business apps, GUIs)

Functional	Data transformation pipelines, concurrent processing, symbolic maths
Declarative	Database queries, rule-based inference, constraint solving
Event-driven	GUIs, games, real-time systems reacting to inputs

Exam Tip: Modern languages are *multi-paradigm* — Python supports procedural, OOP and functional styles. In exam scenarios, justify the paradigm by citing a feature that matches the problem: "OOP because the system has many customer-like entities sharing behaviour that can be inherited".

Key Terms Recap

Term	One-line definition
Kernel	Core of the OS running in privileged mode, handling processes, memory, I/O, interrupts
Paging	Fixed-size division of virtual memory mapped to physical frames via a page table
Page fault	Exception raised when a process accesses a page not in physical RAM
ISR	Interrupt Service Routine — code that handles a specific interrupt
Virtual machine	Software emulation of a computer (system VM) or abstract machine executing intermediate code (process VM)
JIT	Just-in-time compilation — translating intermediate code to native at runtime
Compiler	Translator producing target code once; program then runs natively
Interpreter	Translator that reads and executes source line by line
AST	Abstract Syntax Tree — hierarchical output of the parser

Linker	Combines object files and libraries into one executable
Loader	OS component that loads the executable into RAM at run time
Waterfall	Sequential methodology — requirements → analysis → design → implement → test → maintain
Agile	Iterative methodologies delivering working software in short sprints
TDD	Test-Driven Development — write failing test, make it pass, refactor
Addressing mode	How an instruction's operand should be interpreted (immediate / direct / indirect / indexed)
Encapsulation	Bundling data and methods and restricting direct data access
Polymorphism	One interface, many implementations (override / overload)

Worked Scenarios

Scenario: Choosing between a compiler and an interpreter

A development team is building an embedded temperature-control firmware that will ship on thousands of identical microcontrollers with 32 KiB of flash memory and no screen. Which translator should they use?

- A **compiler** is strongly preferred. Embedded flash is tight, so the overhead of shipping an interpreter with each device is unacceptable.
- Native machine code runs faster and uses less power — important for battery-backed sensors.
- The source is proprietary — shipping a compiled binary protects IP.
- The firmware is rarely changed; there is no need for the interactive development benefits of an interpreter.

Scenario: Choosing a methodology for a hospital triage system

A hospital trust is commissioning a new patient triage system. Requirements must be fully documented for regulatory audit, and the system must be highly reliable. The Waterfall or Spiral models are both reasonable: Waterfall if the requirements can be frozen early (documentation and audit trail are natural deliverables); Spiral if there is significant risk of novel integration issues, because each loop explicitly analyses and resolves risks before coding. Agile would be a poor fit on its own because of the light documentation and difficulty of fix-pricing a regulated project.

Scenario: Choosing an OS type for a smart thermostat

A smart thermostat needs to sample temperature every second and drive a boiler relay within 100 ms of a setpoint change. A full desktop OS (Windows, Linux) is overkill and cannot guarantee response time. The correct choice is an **embedded real-time OS** (FreeRTOS, Zephyr) running on a microcontroller: low RAM footprint, deterministic scheduler, interrupts for the button panel, and no unnecessary background processes competing for CPU.

Exam Technique for 1.2

- **Define before discussing.** Any term named in the spec is worth a 1-mark definition; write it before analysing.
- **Name the stages.** For compilation, methodologies and FDE, sequence matters — lost if you swap stages.
- **Contrast pairs.** Questions love to pit compiler vs interpreter, static vs dynamic linking, Waterfall vs Agile, polling vs interrupts. Learn these side-by-side.
- **Apply to scenarios.** Long-answer questions give a context (a start-up, a hospital, a games company). Pick the methodology/paradigm/OS type that matches and justify against scenario cues.
- **Use proper terminology.** "Kernel", "ISR", "AST", "intermediate representation", "abstraction" — examiners reward precise vocabulary.
- **Show the tradeoff.** Most H446 1.2 questions are not asking for "which is best", but for *balanced discussion*. Always give at least one pro *and* one con.
- **Don't confuse layers.** Translators (lexer/parser/...) sit inside a compiler, the compiler is invoked by the toolchain, the toolchain runs on the OS, the OS sits on the hardware. Mixing these layers loses marks.

